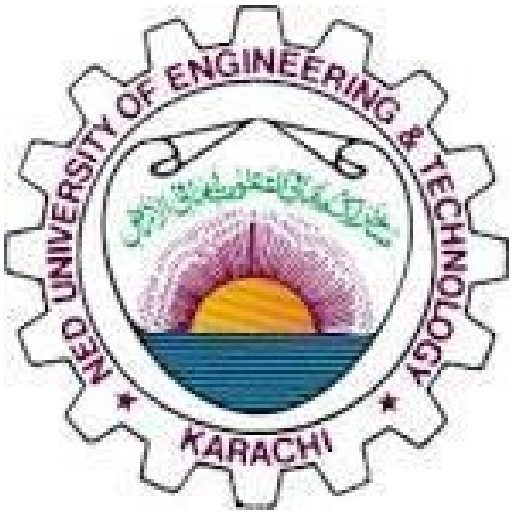
OBJECT ORIENTED PROGRAMMING [CT-260]



NAME: ABDUL RAHEEM SHEIKH

ROLL NO: CT-25192

DEPARTMENT: BCIT

BATCH: 2025

YEAR & SECTION: FSCS-D

LAB: 10

CODES
QUESTION 01

```cpp
#include <iostream>
using namespace std;

template <class T1, class T2>
class Calculator {
    T1 num1;
    T2 num2;

public:
    Calculator(T1 a, T2 b) {
        num1 = a;
        num2 = b;
    }

    void add() {
        cout << "Addition = " << num1 + num2 << endl;
    }

    void subtract() {
        cout << "Subtraction = " << num1 - num2 << endl;
    }

    void multiply() {
        cout << "Multiplication = " << num1 * num2 << endl;
    }

    void divide() {
        if (num2 != 0)
            cout << "Division = " << num1 / num2 << endl;
        else
            cout << "Division by zero not possible!" << endl;
    }
};

int main() {
    Calculator<int, float> c1(20, 5.5);

    c1.add();
```

```
    c1.subtract();
    c1.multiply();
    c1.divide();

    return 0;
}
```

OUTPUT:

```
Addition = 25.5
Subtraction = 14.5
Multiplication = 110
Division = 3.63636
```

QUESTION 02

```
#include <iostream>
using namespace std;

template <class T1, class T2>
void swapData(T1 &a, T2 &b) {
    auto temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 20;
    double y = 14.5;

    cout << "Before Swapping:" << endl;
    cout << "x = " << x << " y = " << y << endl;

    swapData(x, y);

    cout << "After Swapping:" << endl;
    cout << "x = " << x << " y = " << y << endl;

    return 0;
}
```

OUTPUT:

```
Before Swapping:
x = 20 y = 14.5
After Swapping:
x = 14 y = 20
```

QUESTION 03

```cpp
#include <iostream>
using namespace std;

// General template
template <class T>
class mycontainer {
    T element;

public:
    mycontainer(T arg) {
        element = arg;
    }

    T increase() {
        return ++element;
    }
};

// Specialized template for char
template <>
class mycontainer<char> {
    char element;

public:
    mycontainer(char arg) {
        element = arg;
    }

    char uppercase() {
        if (element >= 'a' && element <= 'z')
            element = element - 32;

        return element;
    }
};

int main() {

    mycontainer<int> obj1(3);
    cout << "Increased value: " << obj1.increase() << endl;
```

```
    mycontainer<char> obj2('y');
    cout << "Uppercase value: " << obj2.uppercase() << endl;

    return 0;
}
```

OUTPUT:

```
Increased value: 4
Uppercase value: Y
```

QUESTION 04

```
#include <iostream>
using namespace std;

// Abstract Class
template <class T>
class Array {
public:
    virtual bool isFull() = 0;
    virtual bool isEmpty() = 0;
    virtual int size() = 0;
    virtual T Front() = 0;
    virtual T Rear() = 0;
    virtual void enqueue(T value) = 0;
    virtual void dequeue() = 0;
    virtual void resize() = 0;
};

// Queue Class
template <class T>
class Queue : public Array<T> {
    T *arr;
    int front, rear, capacity, count;

public:
    Queue(int size = 5) {
        capacity = size;
        arr = new T[capacity];
```

```
        front = 0;
        rear = -1;
        count = 0;
    }

    bool isFull() {
        return count == capacity;
    }

    bool isEmpty() {
        return count == 0;
    }

    int size() {
        return count;
    }

    T Front() {
        return arr[front];
    }

    T Rear() {
        return arr[rear];
    }

    void enqueue(T value) {

        if (isFull())
            resize();

        rear = (rear + 1) % capacity;
        arr[rear] = value;
        count++;

        cout << value << " inserted into queue" << endl;
    }

    void dequeue() {

        if (isEmpty()) {
            cout << "Queue is empty!" << endl;
            return;
        }

        cout << arr[front] << " removed from queue" << endl;
```

```
        front = (front + 1) % capacity;
        count--;
    }

    void resize() {

        int newCapacity = capacity * 2;
        T *newArr = new T[newCapacity];

        for (int i = 0; i < count; i++) {
            newArr[i] = arr[(front + i) % capacity];
        }

        delete[] arr;

        arr = newArr;
        front = 0;
        rear = count - 1;
        capacity = newCapacity;

        cout << "Queue resized to " << capacity << endl;
    }
};

int main() {

    Queue<int> q;

    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);

    cout << "Front: " << q.Front() << endl;
    cout << "Rear: " << q.Rear() << endl;

    q.dequeue();

    cout << "Queue Size: " << q.size() << endl;

    return 0;
}
```

OUTPUT:

```
10 inserted into queue
20 inserted into queue
30 inserted into queue
Front: 10
Rear: 30
10 removed from queue
Queue Size: 2
```

QUESTION 05

```cpp
#include <iostream>
#include <queue>
#include <string>
using namespace std;

class PrintJob {
public:
    string jobName;

    PrintJob(string name) {
        jobName = name;
    }
};

int main() {

    queue<PrintJob> printerQueue;

    // Enqueue print jobs
    printerQueue.push(PrintJob("Document1.pdf"));
    printerQueue.push(PrintJob("Assignment.docx"));
    printerQueue.push(PrintJob("Image.png"));

    cout << "Print jobs added to queue.\n" << endl;

    // Process jobs
    while (!printerQueue.empty()) {

        PrintJob currentJob = printerQueue.front();
        printerQueue.pop();

        cout << "Printing: " << currentJob.jobName << endl;
        cout << "Job completed!\n" << endl;
    }
```

```
    cout << "All print jobs completed. Printer is idle." << endl;

    return 0;
}
```

OUTPUT:

```
Print jobs added to queue.

Printing: Document1.pdf
Job completed!

Printing: Assignment.docx
Job completed!

Printing: Image.png
Job completed!

All print jobs completed. Printer is idle.
```